A Fully Asynchronous Parallel-Path Hierarchical NoC for Scalable High-Fanout On-Chip Communication

Anonymous Authors

***Abstract*—As manycore processors, AI accelerators, and other large SoCs grow, their on-chip networks must carry traffic across longer distances and increasingly shared links. This scaling creates two related problems: global traffic competes for a small number of upper-level links, while one-to-many transfers send repeated copies of the same data through those links. The network therefore needs more parallel paths and in-network replication, while avoiding unnecessary copies at the source. We address this problem with a fully asynchronous parallel-path network-on-chip (PROP). Each 64-node tile contains a three-level local tree, while independent 2×2 upper-mesh planes connect the tile boundaries. Between adjacent tree levels, parallel physical lanes carry independent data and handshake channels. Native multicast keeps one packet while destinations share a path and creates copies only when paths diverge. A phase-safe controller binds each packet branch to one lane and preserves the two-phase request/acknowledge protocol during the selection. Post-synthesis evaluation shows that PROP reaches 379.2 Mflit/s per node in the 64-node network, 58.7% above a flat mesh. Native multicast reduces fanout-16 completion latency at 5 Mflit/s per node by 93.8% relative to repeated unicast. In the 256-node four-tile network, PROP reaches 154.7 Mflit/s per node, 37.5% above the flat-mesh baseline.**

***Index Terms*—asynchronous NoC, hierarchical NoC, hardware multicast, multi-lane routing, scale-out interconnects, neuromorphic computing**

I. Introduction

Modern manycore processors, AI accelerators, and other large SoCs integrate hundreds or thousands of compute and memory nodes. Within these chips, the on-chip network determines how efficiently data moves between nodes and tiles. As the node count grows, packets traverse more routers, traffic from many tiles converges on shared links, and one-to-many transfers increase replication traffic. TrueNorth integrates 4,096 neurosynaptic cores [1], PAICORE connects 1,024 cores through a hierarchical fabric [2], and Hala Point scales 1,152 Loihi 2 processors to 140,544 cores [3], [4]; these systems illustrate how communication becomes a limiting factor as on-chip systems scale.

Scale-out creates a path-capacity problem. Local traffic can remain within one tile, but inter-tile traffic must share a smaller set of upper-level links. As the number of nodes increases, these shared links can become congested even when local routers are lightly loaded. A scalable NoC therefore needs several independent links between levels so that traffic can be distributed across multiple paths. The same issue appears in dense streaming and sparse event-driven systems; neuromorphic events provide a clear example because a short event may reach nearby and distant destinations [5]–[7].

Fanout creates a second problem. Sending one-to-many traffic as separate unicast packets repeats the same flit on common links, increasing injection and queueing pressure. Widening one upper-level router can provide more capacity, but it also makes that router handle more competing requests and increases its control cost. The proposed design uses several parallel routers and keeps a multicast packet shared until its destination paths diverge.

Following the standard interconnection-network model [8], [9], communication latency is

$$T_{\text{comm}} = Ht_r + Ht_w + T_c + \frac{L}{b}. \tag{1}$$

where H is hop count, t_r and t_w are the per-hop router and link delays, respectively, T_c is contention delay, and L/b is serialization delay. Topology reduces H and supplies path capacity; routing distributes load and shares multicast paths; router microarchitecture reduces local delay and increases service rate. Existing techniques improve individual terms through high-radix or hierarchical connectivity [2], [10], adaptive routing and hardware multicast [11], [12], or wide and asynchronous routers [13]–[15].

These topology and router choices expose different implementation tradeoffs. Regular mesh and torus networks retain simple routers but require more hops as dimensions grow. High-radix and hierarchical structures reduce hop count or preserve locality, while adaptive routing and hardware multicast distribute load and share common path segments. Wider datapaths and parallel lanes increase service rate, but a hierarchy can concentrate traffic in a few upper-level routers and make them congested. Scale-out therefore requires distributed path capacity, in-network replication, and control that safely coordinates parallel physical lanes.

This paper presents a fully asynchronous parallel-path hierarchical NoC for high-fanout communication. Its contributions are:

- **Parallel-path hierarchy:** multiple intermediate routers connect several physical lanes between stages, while independent mesh planes connect tiles. The structure preserves short local paths and distributes converging traffic.
- **Native hierarchical multicast:** one injected packet remains multicast across the local tree and upper-mesh planes, replicating only when destination paths diverge and sharing common path segments.
- **Phase-safe parallel-lane control:** packet-lifetime lane binding preserves two-phase request and acknowledgment state during lane selection and arbitration without local clocks.

II. Background and Motivation

A. Communication in Large-Scale Systems

Large systems combine local traffic with less frequent global transfers. Address-Event Representation (AER) is one concrete example: an event is a compact address-bearing packet representing one or more neural spikes, and one event may target multiple destination cores [5], [6]. Thus a sparse source can still create high-fanout transfers, routing metadata, and contention [7]. The same distinction appears in accelerator and manycore systems as local streams, regional synchronization, and global

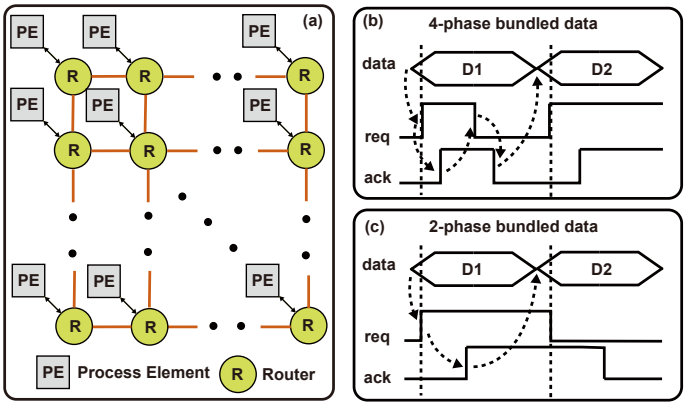


Fig. 1. Basic communication concepts for scalable asynchronous NoCs: (a) PE/router connectivity, (b) four-phase bundled-data transfer, and (c) two-phase bundled-data transfer.

reductions. Local traffic therefore needs short paths and low per-hop overhead, while global and high-fanout traffic needs aggregate bisection capacity and efficient replication.

The interconnect determines whether this traffic can scale. A shared bus is simple, but every transfer passes through one arbitration point and one wire bundle; serialization, electrical fanout, and wire length limit bandwidth as the number of PEs increases. A NoC distributes point-to-point links and routers, so independent transfers can reuse different links concurrently [8], [9]. Fig. 1(a) shows this organization: PEs attach to local routers, which make capacity, locality, and path diversity explicit design dimensions.

In this paper, a *link* is the physical connection between adjacent routers, a *lane* is an independently handshaken data channel on one logical direction, and a *path* is the multi-hop route followed by a packet.

B. Scale-Out NoC Architectures

Topology, routing, and router microarchitecture then determine complementary parts of communication cost. Meshes are modular but their diameter grows with system size; high-radix and hierarchical networks shorten paths or exploit locality at the cost of wiring and arbitration [2]. Multicast shares common path segments through in-network replication, while bypass, parallel datapaths, and optimized arbitration improve service capacity [6], [7], [9], [13], [16], [17].

Asynchronous signaling supplies the timing discipline for these links. In bundled-data communication, the sender stabilizes the data and changes a request; the receiver samples the data and returns an acknowledgment. Fig. 1(b) shows four-phase signaling, in which both control signals return to zero before the next transfer. Fig. 1(c) shows the two-phase variant used here: request and acknowledgment transitions delimit successive transfers without that reset interval. Two-phase links therefore advance on local completion, avoid a network-wide clock, and better tolerate router and wire-delay variation [15], [18], [19]. This reduces per-hop protocol overhead while allowing topology and routing to scale independently. It is also the protocol basis for the phase-safe lane binding and PhaseAdapter described in Section III-C; the handshake complements, rather than replaces, path capacity.

Deterministic routing provides predictable paths, whereas adaptive routing uses path diversity to avoid heavily loaded regions. Repeated unicast sends the same multicast data over common links; in-network replication removes this redundancy, but its benefit remains limited by physical path capacity. Neither router-level optimization nor two-phase signaling can compensate for insufficient end-to-end capacity.

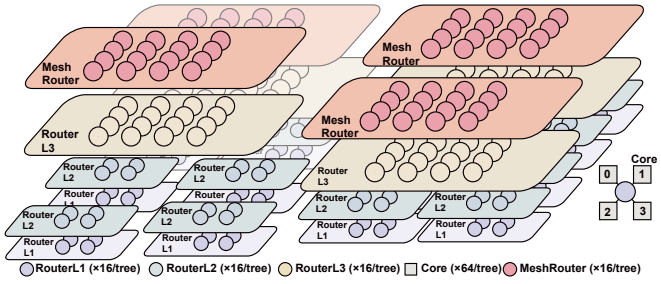


Fig. 2. Proposed 256-node organization.

Hierarchical designs are attractive because they confine nearby traffic, but conventional aggregation progressively reduces the number of upward resources. Increasing the lane count of a single upper router can recover bandwidth, yet its arbitration structures must handle more competing requests. The proposed design spreads the upward links across several routers. Each router handles a limited number of competing requests, while the combined structure still provides multiple paths through the hierarchy.

C. Design Motivation

No single layer resolves scale and fanout together: hierarchy may concentrate traffic, multicast still needs physical capacity, and faster routers do not remove long routes. The proposed design therefore combines distributed parallel connectivity, cross-level multicast, and phase-safe binding of packet branches to physical lanes (Section III).

The resulting objective is a fixed-depth organization that preserves short local routes without progressively collapsing traffic onto a few wide upper-level routers. Multicast should remain a single packet until a routing decision requires replication, and each branch must retain a stable physical lane for its packet lifetime. In an asynchronous network, that binding must also preserve request and acknowledgment phase state across dynamically selected paths. These requirements jointly motivate the topology, routing, and control mechanisms described next.

III. Proposed NoC

A. Parallel-Path Hierarchical Topology

Fig. 2 shows four identical 64-node tiles. Within each tile, 64 nodes feed 16 L1 routers, followed by 16 L2 and 16 L3 routers and 16 tile-boundary mesh routers. Corresponding mesh routers from the four tiles form 16 independent 2×2 upper-mesh planes. The fixed-depth hierarchy keeps local traffic on short tree paths, while replicated upper paths distribute converging and inter-tile traffic. All We denote a router with X child-side lanes and Y parent-side lanes per logical direction as cXpY. All four router levels use the c1p4 geometry—one child-side lane and four parent-side lanes per logical direction—so replication raises aggregate path capacity without concentrating all requests in one wider router.

Fig. 3(a) and (b) expand the two local interconnection stages. Within each quadrant q, the four L1 routers $L1(q, i)$ are fully connected to four replicated L2 routers $L2(q, k)$. Each L1 can therefore select any of four upward L2 paths, while every L2 can receive from all four L1 groups in the quadrant. The second stage regroups routers by k: for each k, the four $L2(q, k)$ routers from different quadrants connect to four $L3(j, k)$ routers. An upward packet first selects k at L1–L2 and then j at L2–L3, yielding $4 \times 4 = 16$ physical path combinations. These replicated paths retain the logical tree hierarchy and its region-based routing semantics while distributing traffic across separate routing elements.

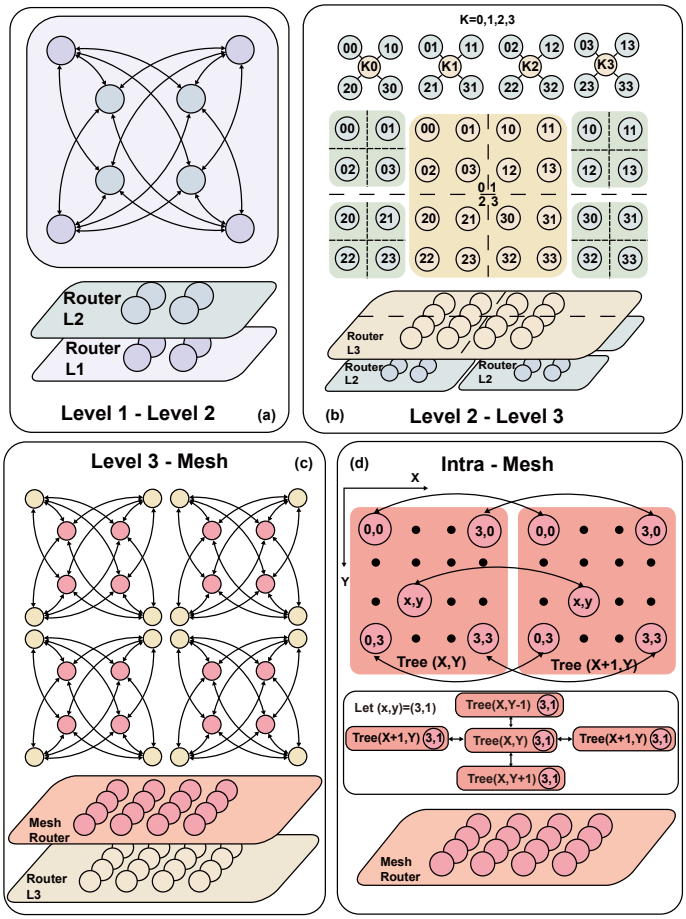


Fig. 3. Inter-level connectivity and upper-mesh planes.

Rather than further extending the hierarchy with increasingly wide upper-level routers, the local tree terminates at L3 and distributes global traffic over multiple upper-mesh planes. Fig. 3(c) shows the L3-to-mesh distribution. Let P_h denote the h-th parent lane of an L3 router and L_k the k-th local lane of a tile-boundary mesh router, where $j, h, k \in \{0, 1, 2, 3\}$. The connection is $L3(j,k).P_h \leftrightarrow Mesh(j,h).L_k$. Consequently, the 16 L3 routers expose 64 tile-boundary lanes, distributed evenly among 16 tile-boundary mesh routers; each mesh router receives four L3 lanes through its four local lanes.

Fig. 3(d) shows the intra-mesh step. For a fixed plane index (j,h), the corresponding $Mesh(j,h)$ router from each tile occupies one coordinate of a 2×2 mesh. Links in that plane connect only routers with the same (j,h) index across adjacent tiles. The 256-node configuration therefore contains 16 independent upper-mesh planes and 64 mesh routers in total. This mapping terminates the local tree without introducing another high-radix aggregation point: local communication remains in the fixed-depth hierarchy, while packets that cross tile boundaries continue through one selected upper-mesh plane.

The capacity effect can be expressed with a cut-capacity bound. For a region S containing s nodes in an N-node network under uniform-random unicast traffic, the average traffic leaving S is

$$\Lambda_{\text{out}} = s\lambda \frac{N-s}{N-1}. \tag{2}$$

where λ is the per-node injection rate. If the cut provides c directed lanes of capacity C, the sustainable injection rate is bounded by

$$\lambda \le \frac{cC(N-1)}{s(N-s)}. \tag{3}$$

TABLE I
28-BIT FLIT FORMAT USED BY THE ROUTING LOGIC.

Head	Tail	y_1	x_1	y_0	x_0	ID
[27]	[26]	[25:20]	[19:14]	[13:8]	[7:2]	[1:0]

For the 64-node local tree, the 4-node and 16-node cuts use $c = 4$ and $c = 16$ directed lanes and yield bounds of $1.05C$ and $1.31C$, respectively, both above the node injection capacity C; hence, neither hierarchical cut becomes the intrinsic bottleneck under this traffic model. At 256 nodes, the 16-plane upper mesh provides a central-bisection bound of $0.498C$ using 32 directed lanes, twice that of an equal-scale single-plane 16×16 mesh with 16 directed lanes ($0.249C$). These are structural capacity bounds rather than throughput predictions; measured performance is evaluated in Section IV.

B. Multicast Routing Across Tree and Mesh

A unified rectangular destination representation is used across both routing tiers. Each multicast packet carries $R_d = [x_0, x_1] \times [y_0, y_1]$, where (x_0, y_0) and (x_1, y_1) denote the two opposite corners of the destination region. At each router v, the routing logic determines a logical output mask $D = R(v, R_d, d_{\text{in}})$, where d_{in} is the ingress direction and each bit of D enables one output direction. Physical-lane selection is handled separately by the multi-lane control in Section III-C, allowing the same multicast packet to traverse the tree–mesh hierarchy without destination rewriting or packet regeneration.

Table I summarizes the 28-bit flit format used by the routing logic. Bits [27] and [26] identify Head and Tail flits. On a Head flit, the four 6-bit coordinate fields y_1, x_1, y_0, x_0 encode the two opposite corners of the destination rectangle, while the 2-bit packet-ID field does not participate in routing. A unicast is represented as a degenerate rectangle with identical corners, allowing the same header to support both point-to-point and rectangular multicast traffic.

Routing within the Local Tree: Within the local tree, routing is determined by the spatial relation between the destination region and the region represented by each router. Let R_v denote the region covered by the current router v, and R_v^i the region associated with its i-th child. A child direction i is activated whenever the destination region overlaps that child region, i.e., $R_d \cap R_v^i \neq \varnothing$. Meanwhile, the parent direction is activated only when R_d is not fully contained in R_v, allowing the packet to continue upward until reaching the smallest hierarchy level that covers all destinations. The ingress direction is excluded from the output mask to avoid immediate backtracking. This rule is recursively applied across the three tree levels: L1 resolves destinations at node level, whereas L2 and L3 operate on progressively larger spatial subregions.

Tree routing algorithm
$D \leftarrow \varnothing$
for each child direction i **do**
  if $i \neq d_{\text{in}}$ and $R_d \cap R_v^i \neq \varnothing$ **then**
    $D \leftarrow D \cup \{i\}$
  end if
end for
if $d_{\text{in}} \neq$ parent and $R_d \not\subseteq R_v$ **then**
  $D \leftarrow D \cup \{\text{parent}\}$
end if
return D

Routing within the Upper Mesh: For inter-tile communication, the destination rectangle is projected onto the upper-mesh coordinate space. Let $R_m = [x_L, x_H] \times [y_L, y_H]$, where $x_L = \lfloor \min \frac{(x_0,x_1)}{8} \rfloor$, $x_H = \lfloor \max \frac{(x_0,x_1)}{8} \rfloor$, and similarly for y_L and y_H. Following prior tree-based multicast routing in 2-D meshes [20], a packet outside R_m is first routed toward the nearest corner using deterministic XY routing. Once the packet

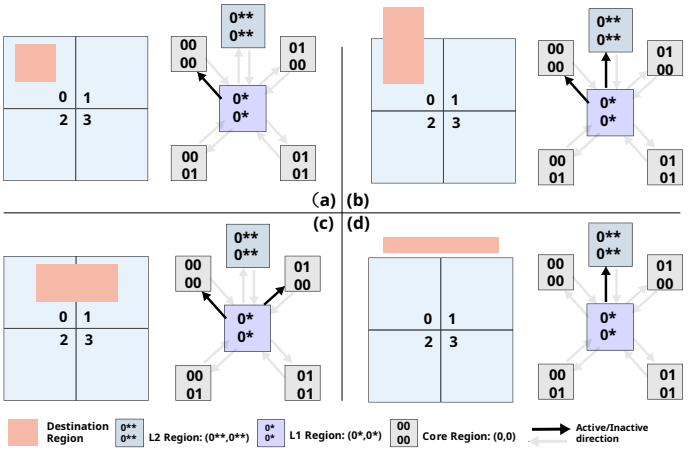


Fig. 4. Local-tree multicast examples for four destination regions. In (a), the destination lies entirely in node region $(0, 0)$, so the packet is sent only to that node. In (b), the destination requires an upward transfer beyond the current L1 region. In (c), the destination covers node regions $(0, 0)$ and $(0, 1)$, so both child directions are enabled. In (d), the destination spans multiple L1 regions, so the packet is sent upward before being replicated at the next level.

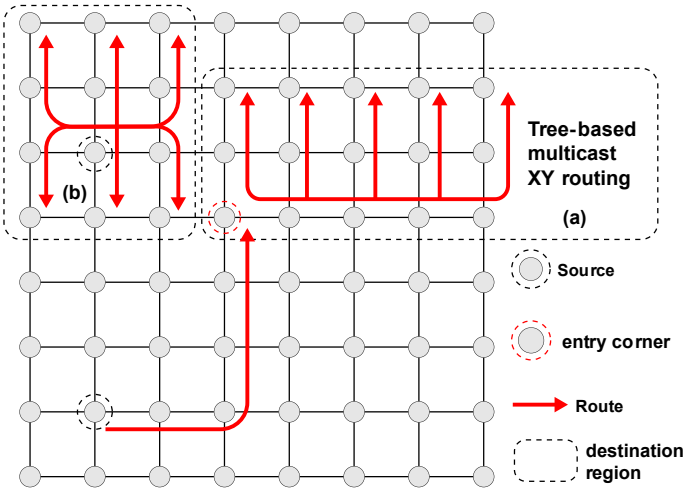


Fig. 5. Upper-mesh multicast examples for two source locations. In (a), the source lies inside the destination region, so the packet is forwarded within that region and branches along the required mesh directions. In (b), the source lies outside the destination region, so the packet first reaches the region through the shared route and then branches inside it. The dashed box marks the destination region and the red arrows show the transmission path.

enters R_m, it is locally delivered and replicated only toward the remaining portions of the destination region according to its ingress direction, as illustrated in Fig. 5. This approach preserves the same destination representation across the tree–mesh boundary and avoids regenerating packets for individual destination tiles.

Mesh routing algorithm

$D \leftarrow \varnothing$; $p \leftarrow (x, y)$
if $p \notin R_m$ **then**
  $q \leftarrow \text{nearestCorner}(p, R_m)$
  $D \leftarrow \{\text{XYNext}(p, q)\}$
else
  if $d_{\text{in}} \neq \text{local}$ **then**
    $D \leftarrow D \cup \{\text{local}\}$
  end if
  $W \leftarrow (x > x_L)$; $E \leftarrow (x < x_H)$
  $N \leftarrow (x_L \le x \le x_H) \wedge (y < y_H)$
  $S \leftarrow (x_L \le x \le x_H) \wedge (y > y_L)$
  if $d_{\text{in}} = W$ **then**
    add E, N, S when their predicates hold
  else if $d_{\text{in}} = E$ **then**
    add W, N, S when their predicates hold
  else if $d_{\text{in}} = N$ **then**
    add S; add W, E only at $y = y_H$
  else if $d_{\text{in}} = S$ **then**
    add N; add W, E only at $y = y_L$
  else
    add every direction whose predicate holds
  end if
end if
remove off-grid directions; **return** D

C. Multi-Lane Arbitration and Phase Adaptation

The input interface module (IPM) receives a bundled-data channel through $ReqIn$, $DataIn$, and the returning $AckOut$, following the CMR organization in [16]. The incoming flits are stored in the five-entry CMR buffer while the route-computation unit decodes the header in parallel. Its PacketPathEnable (PPE) outputs identify every required logical direction and remain active for the packet lifetime. Each enabled read interface then advances an independent read pointer and produces a $ReqDir/DataDir$ stream, allowing multicast branches to progress at different rates. The corresponding $AckDir$ acknowledges each transferred flit, whereas $TailPass$ reports that the selected output has consumed the tail and permits that branch to release its path.

The output interface module (OPM) merges requests for one physical output. Candidate IPMs present PPE_i, Req_i, and $Data_i$, as shown in Fig. 6. A mutual-exclusion arbiter grants one candidate for the packet lifetime; the granted request latch and data multiplexer drive $ReqOut$ and $DataOut$ toward the downstream channel. $AckIn$ is the downstream acknowledgement. After each output transfer, it is returned only as the selected Ack_i, and detection of the transmitted tail generates the matching $TailPass$.

Thus, the IPM decides which directions participate in a multicast, while each OPM independently arbitrates and forwards one lane output stream.

Two arbitration levels are used. In the first level shown in Fig. 7, an asserted PPE allows every available $LaneIsEmpty_i$ to set a $RequestHold_i$. These held requests enter the exact-width mutex, which produces a one-hot $LaneSelect$. Feeding the selected bit back to the mutex input prevents the choice from changing when lane availability changes; the selection is released only when PPE falls at packet completion. In the second level, the OPM on that

TABLE II
ROUTER-LEVEL COMPARISON WITH REPRESENTATIVE ASYNCHRONOUS NOC ROUTERS

Design	Technology	Flit Width	Latency (ns)	Throughput[a] (Gb/s)	Energy[b] (pJ/bit)	Area (μm^2)
CMR [16]	45 nm	128	0.693[c]	–	0.118[e]	54,083
[21]	40 nm	32	0.980[d]	34.24	0.120	24,866
[22]	55 nm	4	1.250[d]	4.00	0.370	22,400
This work (Async c1p4)	**28 nm**	**28**	**1.101**[c]	**84.08**	**0.0545**	**12,602**
This work (Sync c1p4)	**28 nm**	**28**	**1.000**[c]	**69.99**	**0.2254**	**11,374**

[a]Aggregate router throughput. [b]Energy per bit from time-based PrimeTime PX power and throughput. [c]Head-flit/head-hop latency. [d]Reported average forward/flit latency. [e]CMR is derived from 75.4 pJ/(5×128 bits); TaBuLA and Zeng report their values directly.

physical lane arbitrates among competing IPMs and retains its winner until the tail passes. Lane choice and output contention are therefore resolved by separate arbitration points.

The Phase Adapter in Fig. 7 preserves two-phase handshake state across this dynamic binding. Each physical lane owns an independent $OPMReqIn_i$ toggle, while the logical IPM branch owns one $IPMAckIn$ toggle. An $IPMReqOut$ event is translated only to the selected lane. The adapter records when that lane has actually entered the pending state $OPMReqIn_i \neq OPMAckOut_i$, then accepts completion only from the same selected lane and toggles $IPMAckIn$. This ordering keeps lane-selection transients outside the acknowledgement event path and prevents a stale lane phase from creating a premature or missing completion.

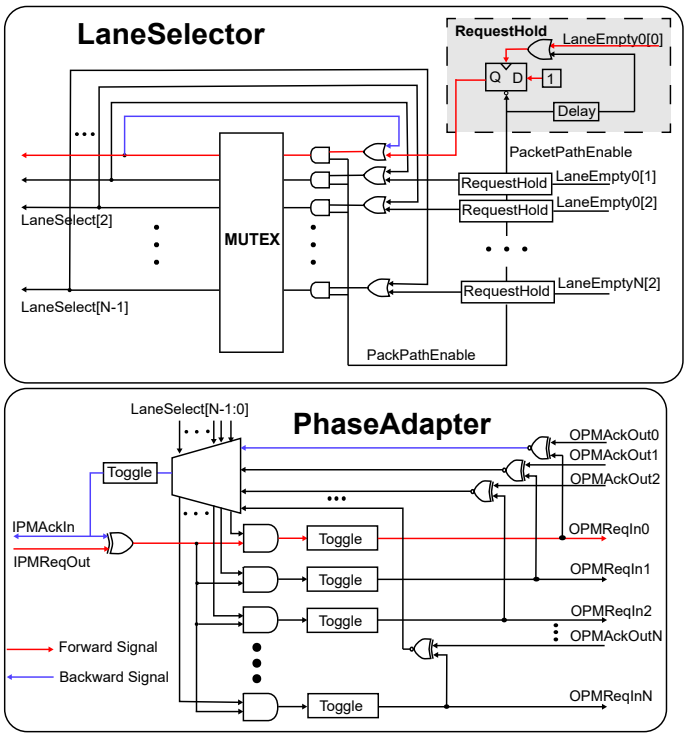


Fig. 7. Packet-lifetime lane selection and phase adaptation.

IV. Evaluation

A. Experiment Setup

All designs use the TSMC 28-nm HPC+ standard-cell library. Timing and performance are obtained from MAXIMUM-

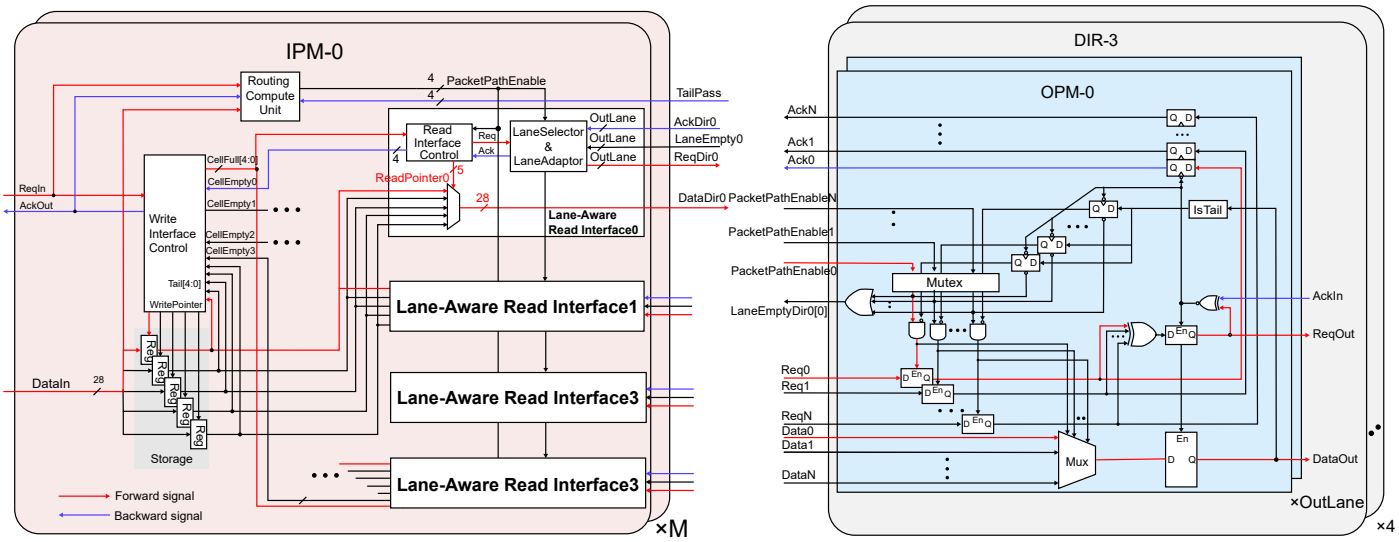


Fig. 6. Internal interconnection of the CMR-based multi-lane router. The input interface computes the route mask and issues requests for eligible outputs. The output interface arbitrates among these requests, binds each packet branch to one physical lane, and forwards the data through the selected child or parent lane. The phase adapter preserves the request and acknowledgment phase relationship across the selected lane.

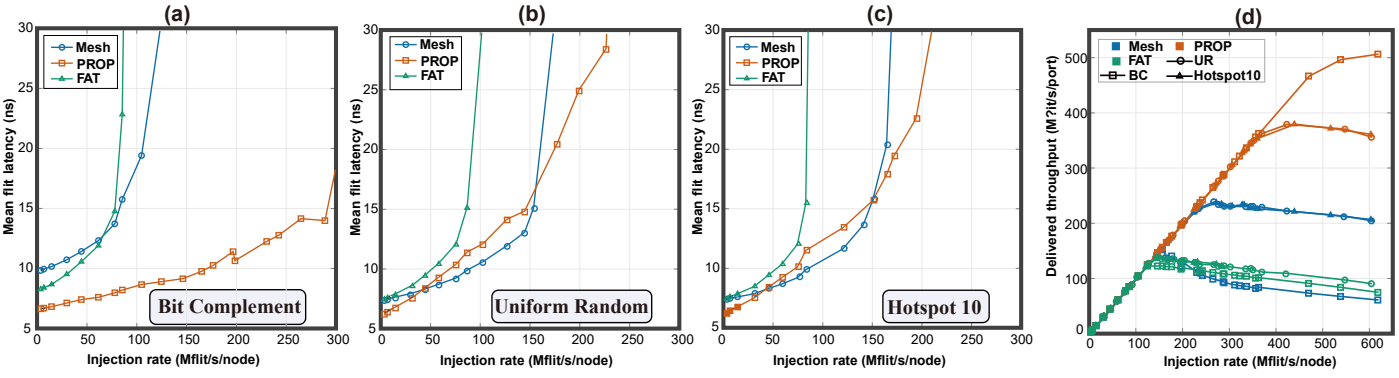


Fig. 8. 64-node unicast evaluation: (a) Bit-Complement mean flit latency, (b) Uniform-Random mean flit latency, (c) Hotspot10 mean flit latency, and (d) throughput versus injection rate for the three patterns.

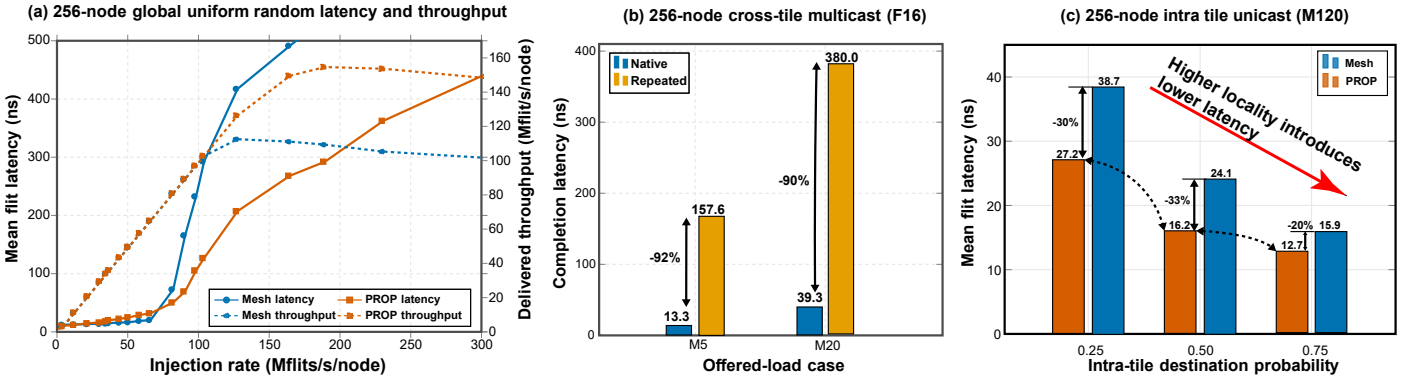


Fig. 9. 256-node scale-out evaluation: (a) global Uniform-Random latency and throughput, (b) cross-tile fanout-16 completion latency, and (c) locality-dependent mean latency.

SDF gate-level simulation; power and energy use time-based PrimeTime PX analysis. All implementation metrics are post-synthesis results.

B. Router-Level Evaluation

Router-level throughput is the aggregate accepted flit rate across all active lanes; head-hop latency is measured from an input request to the corresponding output transfer. For network-level experiments, each case uses five-flit packets. At each source, packet headers are generated with exponentially distributed inter-arrival times. The five flits belonging to one packet are presented as a packet burst, while asynchronous handshakes and router arbitration determine their actual progress through the network. The load labels $M5$, $M20$, and $M100$ denote nominal mean injection rates of 5, 20, and 100 Mflit/s per NoC node, respectively. Since packet arrivals are sampled from an exponential distribution, the measured injection rate of a finite trace can differ slightly from its nominal M value. The label FX denotes a multicast fanout of X; for example, $F16M5$ means fanout 16 at a nominal injection rate of 5 Mflit/s per node.

Table II compares the proposed router with representative asynchronous NoCs. The asynchronous c1p4 router sustains 84.08 Gb/s with a 1.101-ns head-hop latency and 4.581-mW post-synthesis power. Four parent-side lanes increase service capacity while adding lane-selection and arbitration logic; complete-network effects are evaluated next.

C. Network-Level Evaluation

The complete network is evaluated at 64 nodes for topology, control, and multicast ablations, and at 256 nodes for scale-out.

1) 64-Node Ablation Study: The 64-node comparison includes three topologies. Mesh is a flat 8×8 two-dimensional mesh in which each node is attached to one mesh router. FAT is the conventional fat-tree baseline, with progressively wider connectivity toward the root. PROP is the topology proposed in this work: 64 nodes are organized into 16 L1 routers, 16 L2 routers, 16 L3 routers, and parallel tile-boundary mesh routers. The local tree use multiple parent-side lanes to distribute traffic across independent paths, while the tile-boundary mesh routers connect the tile to the upper-mesh planes used for scale-out.

The three unicast patterns exercise different destination distributions. Uniform-Random selects one destination uniformly from the other 63 nodes. Bit-Complement sends each source to the node obtained by complementing its 6-bit node address. Hotspot10 selects from the other 63 nodes while assigning a 1.10 weight to the four central nodes and a 1.00 weight to every other node. Mesh, FAT, and PROP use the same seed, packet length, and generated source-destination trace for each case.

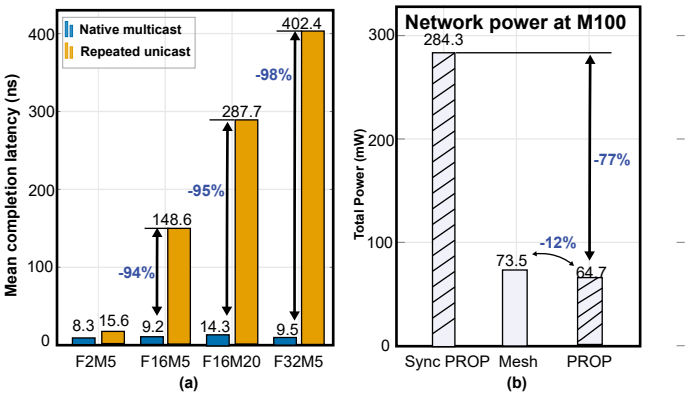


Fig. 10. 64-node results: (a) native multicast versus repeated-unicast completion latency; (b) total network power at $M100$.

TABLE III
64-NODE ASYNC/SYNC RESULTS AT $M100$.

Metric	Async	Sync
Power (mW)	64.7	284.3
Energy (pJ/flit)	13.35	58.66
Mean latency (ns)	10.33	13.35
Area (normalized)	1.116	1.000

Panels (a)–(d) of Fig. 8 show the latency and throughput curves for all three traffic patterns. Under Uniform-Random traffic, PROP reaches 379.2 Mflit/s per NoC node, which is 58.7% higher than Mesh and 2.75× the FAT result. The same ordering appears for Bit-Complement and Hotspot10 traffic, where PROP reaches 506.4 and 379.4 Mflit/s per NoC node, compared with 152.2 and 237.2 for Mesh.

With identical traces, native multicast injects one packet and replicates it within the network only when routes diverge. It reduces completion latency by 94%–98% across F16M5, F16M20, and F32M5, including 9.24 ns versus 148.60 ns at F16M5, as shown in Fig. 10(a). At $M100$, asynchronous PROP uses 64.7 mW and 13.35 pJ/flit, compared with 73.5 mW for Mesh and 284.3 mW for synchronous PROP; its mean latency is 10.33 ns versus 13.35 ns for the synchronous design, with 11.6% area overhead (Fig. 10(b); Table III).

2) 256-Node Scale-Out Evaluation: The 256-node configuration connects four 64-node tiles through 16 independent 2×2 upper-mesh planes and is compared with a 16×16 Mesh. Under global Uniform-Random traffic, identical five-flit traces select destinations uniformly from the other 255 nodes. PROP reaches 154.7 Mflit/s per NoC node versus 112.5 Mflit/s per NoC node for Mesh, a 37.5% increase, and its latency knee occurs at a higher injection rate (Fig. 9(a)).

For fanout-16 traffic split 8+8 across adjacent tiles, native multicast reduces completion latency by 92% at $M5$ and 90%

at $M20$ (Fig. 9(b)). At $M120$, increasing intra-tile locality produces the same trend in both networks, while PROP remains 20%–33% lower in mean latency across the tested probabilities (Fig. 9(c)).

V. Conclusion

This paper presented a fully asynchronous hierarchical NoC for scalable high-fanout communication. The design combines a parallel-path fixed-depth hierarchy, native tree–mesh multicast, and phase-safe multi-lane control to increase path diversity without a global clock or source-side multicast expansion. Post-synthesis results show 379.2 Mflit/s per NoC node peak throughput in the 64-node network (10.6 Gb/s per NoC node at the 28-bit flit width), 58.7% above a flat mesh, while native multicast reduces fanout-16 completion latency by 93.8%. A c1p4 router reaches 3.00 Gflit/s aggregate throughput, and the 256-node four-tile system reaches 37.5% higher peak throughput than a conventional mesh. Together, these measurements support coordinated topology, multicast, and asynchronous control for high-fanout traffic.

References

[1] P. A. Merolla, J. V. Arthur, R. Alvarez-Icaza, A. S. Cassidy, J. Sawada, F. Akopyan, B. L. Jackson, N. Imam, C. Guo, Y. Nakamura *et al.*, "A million spiking-neuron integrated circuit with a scalable communication network and interface," *Science*, vol. 345, no. 6197, pp. 668–673, 2014.

[2] Y. Zhong, Y. Kuang, K. Liu *et al.*, "Paicore: A 1.9-million-neuron 5.181-tsops/w digital neuromorphic processor with unified snn-ann and on-chip learning paradigm," *IEEE Journal of Solid-State Circuits*, vol. 60, no. 2, pp. 651–671, 2025.

[3] Intel Labs, "Taking neuromorphic computing to the next level with loihi 2," Intel Corporation, Tech. Rep., 2021, technology Brief.

[4] Intel Corporation, "Intel builds world's largest neuromorphic system to enable more sustainable ai," Apr. 2024, intel Newsroom, Apr. 17, 2024.

[5] S. Moradi, N. Qiao, F. Stefanini, and G. Indiveri, "A scalable multicore architecture with heterogeneous memory structures for dynamic neuromorphic asynchronous processors (DYNAPs)," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 12, no. 1, pp. 106–122, 2018.

[6] P. Merolla, J. Arthur, R. Alvarez-Icaza, J.-M. Bussat, and K. Boahen, "A multicast tree router for multichip neuromorphic systems," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 61, no. 3, pp. 820–833, 2014.

[7] Z. Su, A. Bencsik, G. Indiveri, and D. Bertozzi, "An efficient multicast addressing encoding scheme for multi-core neuromorphic processors," in *Proc. IEEE International Symposium on Circuits and Systems (ISCAS)*, 2025.

[8] W. J. Dally and B. Towles, *Principles and Practices of Interconnection Networks*. San Francisco, CA, USA: Morgan Kaufmann, 2004.

[9] T. Krishna, C.-H. O. Chen, W. C. Kwon, and L.-S. Peh, "Breaking the on-chip latency barrier using SMART," in *Proc. 19th IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2013, pp. 378–389.

[10] J. Kim, J. Balfour, and W. J. Dally, "Flattened butterfly topology for on-chip networks," *IEEE Computer Architecture Letters*, vol. 6, no. 2, pp. 37–40, 2007.

[11] X. Xiang, W. Shi, S. Ghose, L. Peng, O. Mutlu, and N.-F. Tzeng, "Carpool: A bufferless on-chip network supporting adaptive multicast and hotspot alleviation," in *Proc. International Conference on Supercomputing (ICS)*, 2017.

[12] N. Enright Jerger, L.-S. Peh, and M. H. Lipasti, "Virtual circuit tree multicasting: A case for on-chip hardware multicast support," in *Proc. 35th Annual International Symposium on Computer Architecture (ISCA)*, 2008, pp. 229–240.

[13] T. Fischer, M. Rogenmoser, T. Benz, F. K. Gürkaynak, and L. Benini, "Floonoc: A 645-gb/s/link 0.15-pj/b/hop open-source noc with wide physical links and end-to-end axi4 parallel multistream support," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 33, no. 4, pp. 1094–1107, 2025.

[14] S. Liu, M. Roediger, and A. Karanth, "Monet: A mixture-of-experts accelerator with a multicast-optimized two-tier network-on-chip," in *Proc. Design, Automation and Test in Europe Conference (DATE)*, 2026.

[15] C. Zhang, Y. Hu, and Y. Kang, "A reusable methodology for high-performance interconnects using a standard-cell based asynchronous noc router," in *Proc. Design, Automation and Test in Europe Conference (DATE)*, 2026, pp. 1–7.

[16] K. Bhardwaj and S. M. Nowick, "A continuous-time replication strategy for efficient multicast in asynchronous nocs," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 2, pp. 350–363, 2019.

[17] S. Rasheed, P. V. Gratz, S. Shakkottai, and J. Hu, "Storm: A simple traffic-optimized router microarchitecture for networks-on-chip," in *Proc. 8th IEEE/ACM International Symposium on Networks-on-Chip (NOCS)*, 2014, pp. 176–177.

[18] A. Ghiribaldi, D. Bertozzi, and S. M. Nowick, "A transition-signaling bundled data noc switch architecture for cost-effective gals multicore systems," in *Proc. Design, Automation and Test in Europe Conference (DATE)*, 2013, pp. 332–337.

[19] W. Jiang, D. Bertozzi, G. Miorandi, S. M. Nowick, W. Burleson, and G. Sadowski, "An asynchronous noc router in a 14nm finfet library: Comparison to an industrial synchronous counterpart," in *Proc. Design, Automation and Test in Europe Conference (DATE)*, 2017, pp. 732–733.

[20] L. Wang, Y. Jin, H. Kim, and E. J. Kim, "Recursive partitioning multicast: A bandwidth-efficient routing for networks-on-chip," in *Proc. International Symposium on Networks-on-Chip (NOCS)*, 2009, pp. 64–73.

[21] D. Bertozzi, G. Miorandi, A. Ghiribaldi, W. Burleson, G. Sadowski, K. Bhardwaj, W. Jiang, and S. M. Nowick, "Cost-effective and flexible asynchronous interconnect technology for gals systems," *IEEE Micro*, vol. 41, no. 1, pp. 69–81, 2021.

[22] Q. Zeng, J. Wang, J. Cong, and D. Shang, "A lightweight and high-throughput asynchronous message bus for communication in multi-core heterogeneous systems," *IEEE Access*, vol. 12, pp. 48 555–48 568, 2024.